JAVA PROGRAMMING

Beginner to File Handling — Step-by-Step Study Guide

Student Study Roadmap

This document is designed as a permanent study guide for learning Java from the beginning. It follows a structured progression similar to a C Programming learning path, but translates the concepts into Java.

Learning rule: Do not rush. For every topic, understand the meaning, syntax, explanation, example, output, common mistakes and exercise before moving to the next topic.

Complete Learning Roadmap

No.	Topic
1	Introduction to Java
2	Java Syntax
3	Java Program Structure
4	Variables
5	Data Types
6	Constants
7	Input and Output
8	Operators
9	Type Casting
10	Comments
11	Decision Making: if, else, else-if, nested if
12	switch statement
13	Loops: for, while, do-while
14	break and continue
15	Methods
16	Method parameters and return values
17	Method overloading
18	Arrays
19	Strings
20	Classes and Objects
21	Constructors
22	Encapsulation
23	Inheritance

No.	Topic
24	Polymorphism
25	Abstraction
26	Exception Handling
27	File Handling
28	Practical Java Projects

1. Introduction to Java

Java is a general-purpose programming language used to build many types of software. It is especially important to understand Java as an object-oriented language.

Goal: understand what Java is, where it is used, and how it differs from C.

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

2. Java Syntax

Java syntax is the set of rules used to write valid Java programmes. Java uses semicolons to end statements and curly brackets to define blocks.

Example:

```
System.out.println("Hello World");
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

3. Java Program Structure

A basic Java programme contains a class and a main method. For beginners, the main method can be treated as the starting point of execution.

Example:

```
public class Main {
public static void main(String[] args) {
System.out.println("Hello World");
}
}
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

4. Variables

A variable is a named storage location used to hold a value. A declaration specifies the data type and variable name; initialisation gives it a value.

Examples:

```
int age = 20;
String name = "Vicent";
double marks = 85.5;
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

5. Data Types

Important Java types include int for whole numbers, double for decimal values, char for one character, boolean for true/false, and String for text.

Examples:

```
int age = 20;
double price = 2500.50;
```

```
char grade = 'A';
boolean passed = true;
String name = "John";
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

6. Constants

Use final when a variable should not be reassigned after its value is set.

Example:

```
final int DAYS = 7;
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

7. Input and Output

System.out.println() displays output. Scanner is commonly used to receive keyboard input.

Example:

```
import java.util.Scanner;

Scanner input = new Scanner(System.in);
System.out.print("Enter your age: ");
int age = input.nextInt();
System.out.println("Age: " + age);
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

8. Operators

Arithmetic operators include +, -, *, / and %. Comparison operators include ==, !=, >, <, >= and <=. Logical operators include &&, || and !.

Example:

```
int a = 10;
int b = 3;
System.out.println(a + b);
System.out.println(a % b);
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

9. Type Casting

Type casting changes a value from one data type to another where appropriate. Java supports widening conversions automatically and explicit casts for narrowing conversions.

Example:

```
double x = 10.5;
int y = (int) x;
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

10. Comments

Comments explain code and are ignored during execution.

Examples:

```
// Single-line comment

/* Multi-line
comment */
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

11. Decision Making

Use if, else if and else when a programme must make a decision based on a condition.

Example:

```
if (marks >= 50) {
System.out.println("Pass");
} else {
System.out.println("Fail");
}
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

12. switch Statement

switch is useful when one expression is compared with several possible cases.

Example:

```
switch (day) {
case 1:
System.out.println("Monday");
break;
default:
System.out.println("Invalid");
}
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

13. Loops

Loops repeat code. The main beginner loops are for, while and do-while.

Example:

```
for (int i = 1; i <= 5; i++) {
System.out.println(i);
}
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

14. break and continue

break stops a loop or switch. continue skips the current loop iteration and continues with the next iteration.

Example:

```
for (int i = 1; i <= 10; i++) {
if (i == 5) break;
System.out.println(i);
}
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

15. Methods

A method is a named block of code that performs a task. Java commonly calls what C programmers call functions methods.

Example:

```
static int add(int a, int b) {
return a + b;
}
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

16. Parameters and Return Values

Parameters allow data to be passed into a method. A return statement sends a value back to the caller.

Example:

```
static int square(int n) {
return n * n;
}
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

17. Method Overloading

Method overloading allows methods with the same name but different parameter lists.

Example:

```
static int add(int a, int b) { return a + b; }
static double add(double a, double b) { return a + b; }
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

18. Arrays

An array stores multiple values of the same type. Java array indexes start at 0.

Example:

```
int[] numbers = {10, 20, 30, 40, 50};
System.out.println(numbers[0]);
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

19. Strings

String is used for text. Common operations include length(), toUpperCase(), toLowerCase(), charAt() and comparisons using appropriate String methods.

Example:

```
String name = "Vicent";
System.out.println(name.length());
System.out.println(name.toUpperCase());
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

20. Classes and Objects

A class defines the structure and behaviour of objects. An object is an instance created from a class.

Example:

```
class Student {
String name;
int age;
}

Student s1 = new Student();
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

21. Constructors

A constructor is used when an object is created and is commonly used to initialise object data.

Example:

```
class Student {
String name;

Student(String name) {
this.name = name;
}
}
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

22. Encapsulation

Encapsulation protects object data by controlling access, commonly using private fields and public methods such as getters and setters.

Example:

```
class Student {
private int age;

public void setAge(int age) {
this.age = age;
}
}
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

23. Inheritance

Inheritance allows one class to acquire properties and methods from another class using extends.

Example:

```
class Animal {
void sound() { System.out.println("Sound"); }
}

class Dog extends Animal { }
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

24. Polymorphism

Polymorphism allows the same method call to behave differently depending on the object or implementation.

Example:

```
Animal a = new Dog();
a.sound();
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

25. Abstraction

Abstraction hides unnecessary implementation details and focuses on what an object should do. Java supports abstraction through abstract classes and interfaces.

Example:

```
abstract class Animal {
abstract void sound();
}
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

26. Exception Handling

Exception handling allows a programme to deal with runtime problems without stopping unexpectedly. Important keywords include try, catch, finally, throw and throws.

Example:

```
try {
int x = 10 / 0;
} catch (ArithmeticException e) {
System.out.println("Cannot divide by zero");
}
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

27. File Handling

File handling is the final major target in this learning path. You will learn how to create, write, read, append, inspect and delete files, and how to handle file-related exceptions.

Creating and writing example:

```
import java.io.FileWriter;

FileWriter writer = new FileWriter("students.txt");
writer.write("John\n");
writer.close();
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

28. Practical Java Projects

After reaching file handling, combine variables, input, conditions, loops, methods, arrays, OOP, exceptions and files into complete projects.

Suggested project:

```
Student Records System: enter student details, validate data, save records to a file, read
records back, display them, and handle errors safely.
```

Study checklist: Understand the meaning → learn the syntax → type the example yourself → run it → observe the output → modify it → complete an exercise.

C → JAVA Quick Translation Guide

C Programming	Java
printf()	System.out.print() / System.out.println()
scanf()	Scanner input methods
function	method
struct	class (Java OOP approach)
const	final
int array[5]	int[] array
FILE* / fopen / fprintf / fscanf	Java file APIs such as File, FileWriter, BufferedReader, Scanner, Files
try/catch (C has limited equivalent patterns)	try/catch/finally/throw/throws

Personal Study Strategy

1. Study one lesson at a time. 2. Type every example instead of only reading it. 3. Change values and test what happens. 4. Keep a notebook of errors and corrections. 5. Do exercises without looking at the solution first. 6. Build small projects after each group of topics. 7. Do not move to File Handling until you are comfortable with the preceding fundamentals.

End Goal

By the end of this roadmap, you should be able to write Java programmes from scratch, explain the syntax to another beginner, use OOP concepts, handle exceptions, and build practical programmes that store and retrieve information using files.